

PulseGrid AI

Real-Time Emergency Triage & Healthcare Load Optimization System

Technical Architecture & Implementation Blueprint

1. Executive Summary

South African emergency healthcare systems frequently suffer from cascading failures due to hospital overloading, delayed triage, and reactive (rather than predictive) resource distribution. **PulseGrid AI** is an intelligent coordination platform designed to address these systemic bottlenecks.

Moving beyond standard "AI symptom checkers," PulseGrid AI operates as an advanced routing and optimization engine. It utilizes predictive severity scoring combined with an **Accumulative Priority Queueing** mathematical model to dynamically route patients, balance hospital loads, and minimize the Average Time-to-Treatment (ATT) across a distributed healthcare network.

2. Core System Architecture

To ensure scalability and high performance during simulated high-stress events, the system architecture is heavily decoupled. The backend relies on asynchronous processing, while the frontend provides a real-time command dashboard.

Technology Stack Overview

- **Frontend / Visualization:** React.js, TailwindCSS, Recharts (for real-time metrics), Leaflet/Mapbox (for spatial routing).
- **Backend API & Engine:** Python with FastAPI (chosen for native asynchronous support and easy integration with data science libraries).
- **Algorithmic Modules:** SciPy/NumPy (vectorized mathematical operations), Scikit-learn (lightweight classification).
- **State Management:** In-memory structures (for the 72-hour hackathon constraints) coupled with WebSockets for real-time frontend updates.

3. The Algorithmic Core: APQ-h Engine

The standout technical achievement of PulseGrid AI is the rejection of standard First-In-First-Out (FIFO) or static triage queues. Instead, the engine implements **Accumulative Priority Queueing with a Finite Horizon (APQ-h)**.

3.1 Mathematical Model

In standard systems, a lower-urgency patient might wait indefinitely if higher-urgency patients continually arrive. The APQ-h model assigns a dynamically increasing priority score based on the patient's base triage level and their elapsed waiting time.

$$P(t) = U_0 + \alpha \cdot t$$

Where:

- $P(t)$ is the real-time calculated priority score.
- U_0 is the base urgency score generated by the AI upon intake (e.g., Resuscitation = 100, Non-Urgent = 20).
- α is the acceleration factor, determining how fast a patient's priority grows over time.
- t is the elapsed waiting time in minutes.

3.2 The Finite Horizon Constraint

To ensure clinical safety, a finite time horizon H is introduced. If a patient's wait time breaches the absolute maximum safe wait time for their specific category, their priority mathematically spikes to infinity, forcing immediate attention.

$$\text{If } t \geq H_{\text{category}} \rightarrow P(t) = \infty$$

3.3 Python Implementation (Core Logic)

Below is the optimized, standalone Python module that recalculates the priority queue in real-time. This function is called asynchronously by the FastAPI backend to continuously re-sort patients across the hospital network.

```
import time
from typing import List, Dict

TRIAGE_CONFIG = {
    5: {"base_urgency": 100, "alpha": 0.0, "horizon": 0},      # Immediate
    4: {"base_urgency": 80, "alpha": 2.0, "horizon": 10},    # Very Urgent
    3: {"base_urgency": 60, "alpha": 1.0, "horizon": 60},    # Urgent
    2: {"base_urgency": 40, "alpha": 0.5, "horizon": 120},   # Standard
    1: {"base_urgency": 20, "alpha": 0.2, "horizon": 240}    # Non-Urgent
}

def calculate_apqh_priority(patient: Dict, current_time: float) -> float:
    triage_level = patient.get("triage_level", 1)
    arrival_time = patient.get("arrival_time", current_time)

    wait_mins = (current_time - arrival_time) / 60.0
    config = TRIAGE_CONFIG.get(triage_level, TRIAGE_CONFIG[1])

    if config["horizon"] == 0 or wait_mins >= config["horizon"]:
        return float('inf')

    return config["base_urgency"] + (config["alpha"] * wait_mins)

def update_and_sort_queue(patient_queue: List[Dict]) -> List[Dict]:
    now = time.time()
    for patient in patient_queue:
        patient["current_priority"] = calculate_apqh_priority(patient, now)

    return sorted(patient_queue, key=lambda p: p["current_priority"], reverse=True)
```

4. Intelligent Network Routing

Once priority is established, PulseGrid AI evaluates the geographical topology of the city. When dispatching an ambulance or rerouting a patient, the system calculates the **Predicted Time-to-Treatment (PTT)** rather than just geographic distance.

The PTT Routing Algorithm

$$PTT = \text{Graph_Travel_Time}(A^*) + \text{Hospital_Queue_Delay}(APQ-h)$$

Instead of routing a patient to Hospital A (5 mins away, but with a 2-hour queue backlog), the algorithm routes them to Hospital B (15 mins away, but with zero queue). This effectively balances the load across the entire regional grid.

5. Discrete-Event Simulation (DES) Engine

To demonstrate the efficacy of the algorithms to the hackathon judges, a custom simulation engine runs in the background. The DES engine allows the presenter to programmatically inject "stress events" into the system.

Simulation Event	System Trigger	Expected AI Response
Mass Casualty Event	Spawns 50+ localized severe cases instantly.	Reroutes incoming routine traffic away from the nearest trauma center to peripheral hospitals.
Hospital Failure	Takes a hospital offline (capacity = 0).	Triggers dynamic redistribution of all currently en-route ambulances via A* recalibration.
Gradual Overload	Increases overall intake rate by 300%.	APQ-h parameters automatically shift to strict prioritization, preventing catastrophic finite horizon breaches.

6. Execution Plan (72-Hour Sprint)

To ensure a robust final submission for ALGOfest, development must follow a strict, back-to-front integration strategy.

- **Day 1 (Architecture & Logic):** Set up the Python FastAPI environment. Implement the APQ-h queueing logic, the A* routing mocks, and the data schemas using Pydantic. Ensure the core math is flawless.
- **Day 2 (Simulation & API):** Build the DES engine to generate synthetic patient data. Connect the algorithms to REST/WebSocket endpoints. Begin scaffolding the React frontend, focusing entirely on the command dashboard visualization.
- **Day 3 (Integration & Presentation):** Bind the React dashboard to the WebSocket feeds. Implement the split-screen view (Standard FIFO vs. PulseGrid AI). Deploy the API to Render and the frontend to Vercel. Record the 5-minute technical demo video and finalize the Devpost submission.